

UNIwersytet VIZJA • SCHOOL OF COMPUTER SCIENCE & TECHNOLOGIES

Design and Implementation of a Web-Based Student Management System for Educational Institutions



VIZJA UNIVERSITY

LIVE <https://thesms.me>

STUDENT

Maksym Shpak

Index: 45567

SUPERVISOR

Marcin Kacprowicz

Diploma seminar

FIELD OF STUDY

Computer Science

Engineering degree • 2026

Presentation Agenda

- | | |
|--|---|
| <p>01 Problem Statement & Purpose
Spreadsheets, paper registers, no audit trail</p> <hr/> | <p>05 Database Design
6 models, 6 migrations, constraints in the schema</p> <hr/> |
| <p>02 Comparative Analysis
Moodle, USOS, Google Classroom — and the gap</p> <hr/> | <p>06 Key Features
Weighted grades, RBAC, EN/PL, 96 tests</p> <hr/> |
| <p>03 Technology Stack
Django 6, Python 3.12, PostgreSQL 16</p> <hr/> | <p>07 Application Screenshots
Dashboard, student records, mobile layout</p> <hr/> |
| <p>04 System Architecture
Browser › Render TLS › Gunicorn › PostgreSQL</p> <hr/> | <p>08 Deployment & Results
Live on Render + Neon, then what comes next</p> <hr/> |

Problem Statement & Purpose

3 places

spreadsheet, paper register
and email thread

0

audit trail of who entered
or edited a grade

by hand

every average and every
attendance total

THE PROBLEM

- **Fragmented records** — spreadsheets, paper registers, email threads
- **No single view** — grades and attendance never meet in one place
- **Manual arithmetic** — averages recalculated by hand, errors slip through
- **No accountability** — nobody can prove who recorded a mark
- **No confidentiality** — every teacher can see every student
- **No Polish interface** — staff work in English-only tools

THE PURPOSE

- **One source of truth** — a relational schema with enforced constraints
- **Instant analytics** — KPI tiles and three charts from ORM aggregation
- **Automatic authorship** — every grade stores who assigned it
- **Scoped visibility** — teachers see their own courses, admins see all
- **Bilingual by design** — the whole interface switches EN $\rightleftharpoons$ PL
- **Deployable** — one build script, HTTPS, managed PostgreSQL

Result: one auditable, role-aware platform now managing 25 students, 8 courses, 87 enrollment records — 51 of them active — and 87 grades in production.

Comparative Analysis

CRITERION	MOODLE	USOS	GOOGLE CLASSROOM	SMS — THIS PROJECT
Target user	Large universities	Polish public HEIs	Schools and courses	Departments and small institutions
Deployment	Self-hosted, heavy	Vendor-managed	Google cloud only	Self-hosted, one Django app
Setup effort	Days of plugin work	Institutional contract	Minutes, no control	Minutes — a single build script
Gradebook	Yes, complex	Yes	Basic	Weighted components + author audit
Attendance register	Plugin required	Yes	Not available	Built in, three statuses
Role isolation	Configurable, intricate	Yes	Limited	Enforced at queryset level
Bilingual EN / PL	Language packs	Polish-first	Yes	Native, 312 strings
Analytics	Reports plugin	Institutional reports	Minimal	Chart.js dashboard + CSV
Cost of ownership	GPL + hosting	Licensed	Free, data in Google	Open source, €0 hosting

Positioning: the established platforms are either too heavy or too closed for a single department; SMS trades breadth for a focused, auditable and fully self-hosted core.

Technology Stack

BACKEND

Runtime · data · authentication

- Python 3.12
- Django 6.0.6
- PostgreSQL 16 (Neon)
- SQLite 3 for development
- Django session authentication
- Custom user model with roles
- Django ORM · 6 migrations
- dj-database-url 2.1

FRONTEND

Templates · charts · theming

- Django Templates · 30 files
- Semantic HTML5
- CSS custom properties · 1 945 lines
- Vanilla JavaScript ES6 · 425 lines
- Chart.js 4 — doughnut and bars
- Turbo 8 page transitions
- Bootstrap Icons
- Light and dark themes

INFRASTRUCTURE

Hosting · operations · CI

- Render web service
- Neon managed PostgreSQL
- Gunicorn 22 (WSGI)
- WhiteNoise 6.6 static files
- HTTPS with HSTS preload
- GitHub Actions CI
- Git version control
- Zero CDN dependencies

No build step, no separate API server. Every asset is vendored locally, so the application runs and demonstrates fully offline.

System Architecture

CLIENT

Browser

HTML · CSS · ES6

Turbo navigation



EDGE

Render · TLS 443

HTTPS, HSTS

static via WhiteNoise



APPLICATION

Gunicorn + Django 6

URL › View › Template

ORM aggregation



DATA

PostgreSQL 16

Neon managed

DATABASE_URL, one codebase

REQUEST PIPELINE — MIDDLEWARE

Security

Session

CSRF

Locale

WhiteNoise

Authentication

Role decorators

Queryset scoping

SUPPORTING SERVICES

i18n EN / PL

Interface language switch

GitHub Actions

Tests and deploy gates

Teachers screen

Admin issues lecturer accounts

CSV export

Role-scoped reporting

/api/dashboard/

JSON metrics endpoint

Database Design

User *extends AbstractUser*

id
username • email
first_name • last_name
role → admin | teacher
password (PBKDF2)
is_active

Student *academic identity*

id
student_number (unique)
first_name • last_name
email
study_program
date_enrolled • is_active

Course *taught unit*

id
course_code (unique)
course_name • description
semester • credits
max_students
teacher_id → User

Enrollment *student ↔ course*

id
student_id → Student
course_id → Course
enrollment_date
status → active | completed | dropped
unique (student, course)

Grade *weighted component*

id
enrollment_id → Enrollment
kind → coursework | midterm | final | retake
weight 1-100 %
grade_value 2.0 – 5.0
assigned_by → User

Attendance *daily register*

id
enrollment_id → Enrollment
date
status → present | absent | late
notes
unique (enrollment, date)

Integrity where it belongs: unique constraints, foreign keys and cascade rules live in the schema, so no interface path can create a duplicate enrollment or an orphaned grade.

Key Features

Academic Records

CRUD for students, courses and enrollments.
Administrators issue lecturer accounts on Teachers.
Enrollment is refused once max_students is reached.

Weighted Gradebook

Coursework, midterm, exam and retake — each with a weight. The course mark is their weighted mean. assigned_by records who last saved the component.

Attendance Register

Present, absent and late — one row per enrollment per day, enforced in the schema. Mark Class saves a whole roster in a single request.

Analytics & Export

Four KPI tiles, three Chart.js views and JSON at /api/dashboard/. CSV export of students and grades is scoped to the caller's role.

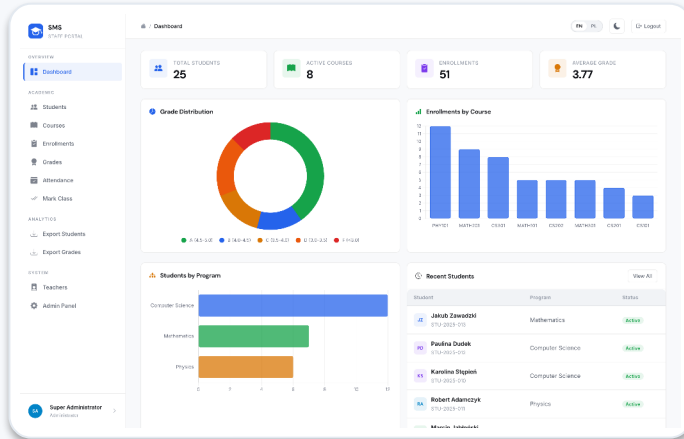
Role-Based Access

Administrators see the institution; teachers see only their courses. Decorators guard the route, querysets filter again — a guessed URL returns nothing.

i18n & Quality

312 strings, English $\rightleftharpoons$ Polish, locale-aware dates. 96 automated tests and four CI gates (tests, migrations, deploy check, collectstatic) on every push.

Application Screenshots



Dashboard — KPI tiles and three Chart.js views

The student register displays a list of students with search and filter capabilities. The table includes columns for Student, Student Number, Program, Enrolled, Courses, Avg. Grade, Status, and Actions. The data is paginated, showing 10 students per page.

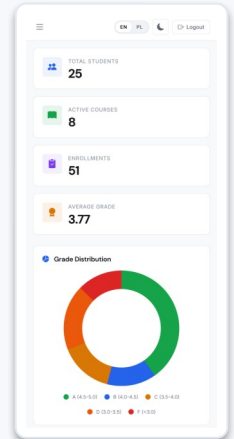
Student	Student Number	Program	Enrolled	Courses	Avg. Grade	Status	Actions
Robert Adamczyk	STU-2025-001	Physics	08/08/2025	3	3.00	Active	[Edit] [Delete]
Paulina Dutk	STU-2025-002	Computer Science	07/10/2025	2	3.75	Active	[Edit] [Delete]
Agnieszka Dąbrowska	STU-2024-003	Mathematics	10/04/2024	1	3.00	Active	[Edit] [Delete]
Natalia Grabowska	STU-2025-004	Physics	10/08/2024	3	3.50	Active	[Edit] [Delete]
Marcin Jablowski	STU-2025-005	Mathematics	03/03/2025	1	2.00	Active	[Edit] [Delete]
Marta Janowska	STU-2024-006	Physics	01/18/2025	2	4.75	Active	[Edit] [Delete]
Piotr Kamiński	STU-2024-008	Computer Science	11/08/2024	2	3.87	Active	[Edit] [Delete]
Alexander Kowalski	STU-2024-009	Computer Science	06/06/2024	4	3.98	Active	[Edit] [Delete]
Krzysztof Kozłowski	STU-2024-011	Computer Science	10/12/2024	2	2.50	Active	[Edit] [Delete]
Łukasz Kowczyk	STU-2025-002	Mathematics	07/27/2025	1	3.25	Active	[Edit] [Delete]
Daniel Krol							

Student register — search, filters, pagination

The Mark Class interface allows for marking attendance for a specific class. It shows a table with columns for Student, Student Number, and Attendance. The data is paginated, showing 10 students per page.

Student	Student Number	Attendance
Robert Adamczyk	STU-2025-001	Present
Natalia Grabowska	STU-2025-004	Absent
Alexander Kowalski	STU-2024-009	Not marked
Katarzyna Liwadowska	STU-2024-008	Present
Adam Mazur	STU-2025-003	Absent
Łukasz Piotrowski	STU-2025-005	Not marked
Michał Wójcik	STU-2024-006	Present
Anna Wójcik	STU-2024-004	Absent
Tomasz Zieliński	STU-2024-007	Not marked

Mark Class — a whole roster saved in one request



Responsive down to 375 px

Every table is role-aware: a teacher opening the same page sees only students in their own courses, and the action buttons disappear.

Deployment & Results

 **LIVE** <https://thesms.me>

DEFENCE LOGIN (teacher)
prof.martinez / demo1234

25

students

8

courses

87

enrollment records

87

grades

€0

monthly cost

DELIVERED

- **Live on Render + Neon** — HTTPS, HSTS, managed PostgreSQL
- **Six-model schema** — constraints and an audit trail in the database
- **Role isolation** — admins and teachers see strictly their own data
- **Bilingual interface** — 312 strings, locale-aware dates and numbers
- **96 tests, four CI gates** — green on every push, then build.sh deploys

FUTURE WORK

- **Student portal** — a third role with read-only access to own results
- **Semester transcripts** — generated PDF reports per student
- **Email notifications** — alerts on new grades and absences
- **Documented REST API** — token-authenticated public interface
- **Two-factor login** — TOTP for administrator accounts

Thank you for your attention

Questions are welcome.

Maksym Shpak

Computer Science • Index: 45567

<https://thesms.me>



VIZJA UNIVERSITY